authorizationManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"authorizationManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `authorizationManagement` service of the ConsumerAuthorization system.

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**
2.1 Interface registration . . . 3

3 Service Operations **4**
3.1 operation grant-policies . . . 4
3.2 operation revoke-policies . . . 5
3.3 operation query-policies . . . 5
3.4 operation check-policies . . . 6

4 Data Models **8**
4.1 struct AuthorizationMgmtGrantListRequest . . . 8
4.2 struct AuthorizationMgmtGrantRequest . . . 8
4.3 struct AuthorizationPolicy . . . 8
4.4 struct AuthorizationPolicyListResponse . . . 8
4.5 struct AuthorizationPolicyRequest . . . 8
4.6 struct AuthorizationPolicyResponse . . . 9
4.7 struct AuthorizationQueryRequest . . . 9
4.8 struct AuthorizationVerifyListRequest . . . 9
4.9 struct AuthorizationVerifyListResponse . . . 10
4.10 struct AuthorizationVerifyRequest . . . 10
4.11 struct AuthorizationVerifyResponse . . . 10
4.12 struct ErrorMessage . . . 10
4.13 struct MetadataRequirement . . . 11
4.14 struct Page . . . 11
4.15 Primitives . . . 11

5 Go and Rust implementation variants **12**
5.1 Go implementation . . . 12
5.2 Rust implementation . . . 12

6 References **12**

7 Revision History **13**
7.1 Amendments . . . 13
7.2 Quality Assurance . . . 13

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the `authorizationManagement` service interface of the ConsumerAuthorization system, which provides operator management of authorization policies at the management level, and bulk evaluation of authorization questions.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `authorizationManagement` service interface

This document provides the Interface Design Description IDD to the *authorizationManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `authorizationManagement` service interface in detail. Both in terms of its operations (Section 3) and its information model (Section 4).

2.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8445 and base path `/consumerauthorization/authorization/mgmt`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`grant-policies`	POST	`/grant`
`revoke-policies`	DELETE	`/revoke`
`query-policies`	POST	`/query`
`check-policies`	POST	`/check`

3 Service Operations

The interfaces of the `authorizationManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `authorizationManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

3.1 POST /consumerauthorization/authorization/mgmt/grant

Operation: grant-policies
Input: AuthorizationMgmtGrantListRequest
Output: AuthorizationPolicyListResponse

Creates management-level policies for the given providers and targets.

A successful call returns `201 Created` with the output in the body.

```
POST /consumerauthorization/authorization/mgmt/grant HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "list": [
    {
      "cloud": "LOCAL",
      "provider": "TemperatureSensor",
      "targetType": "SERVICE_DEF",
      "target": "temperatureReading",
      "description": "Display may read",
      "defaultPolicy": {
        "policyType": "WHITELIST",
        "policyList": [
          "TemperatureDisplay"
        ],
        "policyMetadataRequirement": {
          "unit": "celsius"
        }
      },
      "scopedPolicies": {
        "read": {
          "policyType": "ALL"
        }
      }
    }
  ]
}
```

Listing 1: An grant-policies invocation.

```
{
  "entries": [
    {
```

```
      "instanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "level": "MGMT",
      "cloud": "LOCAL",
      "provider": "TemperatureSensor",
      "targetType": "SERVICE_DEF",
      "target": "temperatureReading",
      "description": "Display may read",
      "defaultPolicy": {
        "policyType": "ALL",
        "policyList": [
          "TemperatureDisplay"
        ],
        "policyMetadataRequirement": {
          "unit": "celsius"
        }
      },
      "scopedPolicies": {
        "read": {
          "policyType": "ALL"
        }
      },
      "createdBy": "Sysop",
      "createdAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 2: An grant-policies response.

3.2 DELETE /consumerauthorization/authorization/mgmt/revoke?instanceIds=*{instanceIds}*

Operation: revoke-policies
Input: Array<String>

Removes policies by instance identifier.
Query parameters: *instanceIds* (List<String>, required). List values are repeated or comma-separated.
A successful call returns `200 OK`.

```
DELETE /consumerauthorization/authorization/mgmt/revoke?instanceIds=<instanceIds> HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An revoke-policies invocation.

3.3 POST /consumerauthorization/authorization/mgmt/query

Operation: query-policies
Input: AuthorizationQueryRequest
Output: AuthorizationPolicyListResponse

Returns a page of policies of either level matching the filter.
A successful call returns `200 OK` with the output in the body.

```
POST /consumerauthorization/authorization/mgmt/query HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
```

```
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "level": "MGMT",
  "providers": [
    "TemperatureSensor"
  ],
  "instanceIds": [
    "TemperatureSensor|temperatureReading|1.0.0"
  ],
  "cloudIdentifiers": [
    "string"
  ],
  "targetNames": [
    "string"
  ],
  "targetType": "SERVICE_DEF"
}
```

Listing 4: An query-policies invocation.

```
{
  "entries": [
    {
      "instanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "level": "MGMT",
      "cloud": "LOCAL",
      "provider": "TemperatureSensor",
      "targetType": "SERVICE_DEF",
      "target": "temperatureReading",
      "description": "Display may read",
      "defaultPolicy": {
        "policyType": "ALL",
        "policyList": [
          "TemperatureDisplay"
        ],
        "policyMetadataRequirement": {
          "unit": "celsius"
        }
      },
      "scopedPolicies": {
        "read": {
          "policyType": "ALL"
        }
      },
      "createdBy": "Sysop",
      "createdAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 5: An query-policies response.

3.4 POST /consumerauthorization/authorization/mgmt/check

Operation: check-policies
Input: AuthorizationVerifyListRequest
Output: AuthorizationVerifyListResponse

Evaluates a list of authorization questions and returns the answer for each.
A successful call returns `200 OK` with the output in the body.

```
POST /consumerauthorization/authorization/mgmt/check HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "list": [
    {
      "provider": "TemperatureSensor",
      "consumer": "TemperatureDisplay",
      "cloud": "LOCAL",
      "targetType": "SERVICE_DEF",
      "target": "temperatureReading",
      "scope": "read"
    }
  ]
}
```

Listing 6: An check-policies invocation.

```
{
  "entries": [
    {
      "provider": "TemperatureSensor",
      "consumer": "TemperatureDisplay",
      "cloud": "LOCAL",
      "targetType": "SERVICE_DEF",
      "target": "temperatureReading",
      "scope": "read",
      "granted": true
    }
  ],
  "count": 1
}
```

Listing 7: An check-policies response.

4 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 4.15, which are used to represent things like hashes, identifiers and texts.

4.1 struct AuthorizationMgmtGrantListRequest

Management-level policies to create.

Object Field	Value Type	Description
"list"	Array<AuthorizationMgmtGrant Request>	The list of request items.

4.2 struct AuthorizationMgmtGrantRequest

A management-level authorization policy for one provider and target.

Object Field	Value Type	Description
"cloud"	String	Cloud identifier of the consumer cloud the policy applies to; `LOCAL` for the own cloud.
"provider"	String	Provider system the policy is set for.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"description"	String	Free-text description.
"defaultPolicy"	AuthorizationPolicyRequest	Policy that applies to all operations without a scoped policy.
"scopedPolicies"	Object<AuthorizationPolicyRequest>	Policies for individual operations, keyed by operation name.

4.3 struct AuthorizationPolicy

One policy rule.

Object Field	Value Type	Description
"policyType"	AuthorizationPolicyType	Policy type: `ALL`, `WHITELIST`, `BLACKLIST` or `SYS_METADATA`.
"policyList"	Array<String>	System names of the white- or blacklist.
"policyMetadata Requirement"	MetadataRequirement	Metadata the consumer system must have (`SYS_METADATA`).

4.4 struct AuthorizationPolicyListResponse

List of authorization policies.

Object Field	Value Type	Description
"entries"	Array<AuthorizationPolicy Response>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

4.5 struct AuthorizationPolicyRequest

One policy rule.

Object Field	Value Type	Description
"policyType"	String	Policy type: `ALL`, `WHITELIST`, `BLACKLIST` or `SYS_METADATA`.
"policyList"	Array<String>	System names of the white- or blacklist.
"policyMetadata Requirement"	MetadataRequirement	Metadata the consumer system must have (`SYS_METADATA`).

4.6 struct AuthorizationPolicyResponse

A stored authorization policy.

Object Field	Value Type	Description
"instanceId"	String	Policy instance identifier `<MGMT or PR>|<cloud>|<provider>|<targetType>|<target>`.
"level"	AuthorizationLevel	Policy level: `MGMT` (set by an operator) or `PROVIDER` (set by the provider itself).
"cloud"	String	Cloud identifier of the consumer cloud the policy applies to; `LOCAL` for the own cloud.
"provider"	String	Provider system the policy protects.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"description"	String	Free-text description.
"defaultPolicy"	AuthorizationPolicy	Policy that applies to all operations without a scoped policy.
"scopedPolicies"	Object<AuthorizationPolicy>	Policies for individual operations, keyed by operation name.
"createdBy"	String	Name of the system that created the record.
"createdAt"	DateTime	UTC time of creation.

4.7 struct AuthorizationQueryRequest

Filter for authorization policies.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"level"	String	Policy level to match, `MGMT` or `PROVIDER`.
"providers"	Array<String>	Provider system names to match.
"instanceIds"	Array<String>	Policy instance identifiers to match.
"cloudIdentifiers"	Array<String>	Cloud identifiers to match.
"targetNames"	Array<String>	Target names to match.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.

4.8 struct AuthorizationVerifyListRequest

Authorization checks to evaluate.

Object Field	Value Type	Description
"list"	Array<AuthorizationVerifyRequest>	The list of request items.

4.9 struct AuthorizationVerifyListResponse

Results of authorization checks.

Object Field	Value Type	Description
"entries"	Array<AuthorizationVerifyResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

4.10 struct AuthorizationVerifyRequest

One authorization question: may the consumer use the target (operation) of the provider?

Object Field	Value Type	Description
"provider"	String	Provider system; for `verify` defaults to the requester.
"consumer"	String	Consumer system; for `verify` defaults to the requester.
"cloud"	String	Cloud of the consumer, `LOCAL` by default.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"scope"	String	Operation to check; empty checks the default policy.

4.11 struct AuthorizationVerifyResponse

The answer to one authorization question.

Object Field	Value Type	Description
"provider"	String	Name of the provider system.
"consumer"	String	Name of the consumer system.
"cloud"	String	Cloud of the consumer.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"scope"	String	The checked operation.
"granted"	Boolean	Result of the check: true when the consumer is authorized.

4.12 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

4.13 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

4.14 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

4.15 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
AuthorizationLevel	One of `MGMT`, `PROVIDER` (JSON String).
AuthorizationPolicyType	One of `ALL`, `WHITELIST`, `BLACKLIST`, `SYS_METADATA` (JSON String).
AuthorizationTargetType	One of `SERVICE_DEF`, `EVENT_TYPE` (JSON String).
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

5 Go and Rust implementation variants

5.1 Go implementation

Primary system folder: `Eclipse-Arrowhead-go/ah5-core-go/consumer-authorization`.

The system owns its Go module, launcher and build configuration. Use the per-system `Makefile` and `go.mod`; the containing Go tree is the build source, not a historical Arrowhead copy. Ordinary native roles use independent identities and JSON snapshot state rather than the Java database. Do not assume Java SQL migration, clustering or identical transaction/retry semantics. The registry has its own configuration and optional native AMQP monitor; other roles must be checked against their own registrations.

Listening port. Built-in default 127.0.0.1:8445 (Arrowhead default). The shipped config.json sets "listen": "0.0.0.0:8445". Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. Native design/route evidence: `Eclipse-Arrowhead-go/ah5-core-go/consumer-authorization/documentation`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

5.2 Rust implementation

Primary system folder: `Eclipse-Arrowhead-rust/ah5-core-rust/consumer-authorization`.

The per-system entry point is built by the `arrowhead-core-rust` crate. Build from the primary Rust workspace with the applicable package/binary selection. Documentation links can resolve into the shared core/support catalogue. Independent process identity and state remain required.

The ordinary native profile uses Tokio/Axum and rustls with peer-identity checks and independent locked JSON state. Network effects are not distributed database transactions. The precise trust, CRL reload and persistence guarantees are those of the native profile, not the Java implementation.

Native design/route evidence: `Eclipse-Arrowhead-rust/ah5-core-rust/documentation/ConsumerAuthorization`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

6 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “authorizationManagement – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.3.0, 2026, consumerAuthorization-v5.3.0-authorizationManagement-SD.pdf.

7 Revision History

7.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

7.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	